Theory Discussion Guide — Interview Preparation (Part A)

This guide covers all topics from Section 3 of the test document. Use it to prepare clear, concise explanations for the interview.

3.1 FDTD-Based EM (Electromagnetic) Forward Modeling

Purpose of FDTD

FDTD (Finite-Difference Time-Domain) directly discretizes Maxwell's curl equations in both space and time. It converts the continuous PDE (Partial Differential Equation) system into explicit algebraic update equations that are marched forward in time step by step. Key advantages over frequency-domain methods:

- **Wideband**: A single simulation covers all frequencies in the source bandwidth
- **Handles complex media**: Inhomogeneous, lossy, dispersive, nonlinear materials
- **Intuitive**: The fields evolve physically in time — easy to visualize and debug
- **Scalable**: Embarrassingly parallel for GPU (Graphics Processing Unit) acceleration

The Yee Grid (1966)

The fundamental innovation by Kane Yee: $\mathbf{E}$ and $\mathbf{H}$ field components are **staggered** in space by half a grid cell and in time by half a time step.

For 2D TMz (Transverse Magnetic with E_z polarization):

Component	Grid location
E_z	integer grid points $(i,\, j)$
H_x	half-step in z-direction $\left(i,\, j+\frac{1}{2}\right)$
H_y	half-step in x-direction $\left(i+\frac{1}{2},\, j\right)$

Why staggering matters:

1. Central-difference approximations become **second-order accurate** automatically
2. The divergence conditions ($\nabla \cdot \mathbf{B} = 0$, $\nabla \cdot \mathbf{D} = \rho$) are **satisfied implicitly**
3. Combined with leapfrog time stepping, the scheme is **dissipation-free** (energy-conserving in lossless media)

Main Time-Stepping Loop (Leapfrog)

Each time step alternates between $\mathbf{H}$ and $\mathbf{E}$ updates:

$$\text{for } n = 0 \text{ to } N_t:$$

1. Update $\mathbf{H}$ from $\mathbf{E}$: $\mathbf{H}^{n+1/2}$ from $\mathbf{E}^n$ — $\mathbf{H}$ jumps half a step forward
2. Apply CPML (Convolutional Perfectly Matched Layer) corrections to $\mathbf{H}$
3. Update $\mathbf{E}$ from $\mathbf{H}$: $\mathbf{E}^{n+1}$ from $\mathbf{H}^{n+1/2}$ — $\mathbf{E}$ jumps one step forward
4. Apply CPML corrections to $\mathbf{E}$
5. Inject source into E_z
6. Record E_z at receiver

The leapfrog structure means $\mathbf{E}$ and $\mathbf{H}$ are never known at the same instant — they interleave in time.

Update Equations (2D TMz)

Maxwell's equations for TMz (no variation in the y-direction):

$$\frac{\partial H_x}{\partial t} = -\frac{1}{\mu}\frac{\partial E_z}{\partial z}$$

$$\frac{\partial H_y}{\partial t} = \frac{1}{\mu}\frac{\partial E_z}{\partial x}$$

$$\frac{\partial E_z}{\partial t} = \frac{1}{\varepsilon}\left(\frac{\partial H_y}{\partial x} - \frac{\partial H_x}{\partial z}\right) - \frac{\sigma}{\varepsilon}E_z$$

Discretized with central differences on the Yee grid:

$$H_x^{\,n+1/2}[i,j] = H_x^{\,n-1/2}[i,j] - \frac{\Delta t}{\mu\,\Delta z}(E_z^{\,n}[i,j+1] - E_z^{\,n}[i,j])$$

$$H_y^{\,n+1/2}[i,j] = H_y^{\,n-1/2}[i,j] + \frac{\Delta t}{\mu\,\Delta x}(E_z^{\,n}[i+1,j] - E_z^{\,n}[i,j])$$

$$E_z^{\,n+1}[i,j] = C_a \cdot E_z^{\,n}[i,j] + C_b\left(\frac{H_y[i,j] - H_y[i-1,j]}{\Delta x} - \frac{H_x[i,j] - H_x[i,j-1]}{\Delta z}\right)$$

where the coefficients incorporate conductivity (σ) losses:

$$C_a = \frac{1 - \dfrac{\sigma\,\Delta t}{2\varepsilon}}{1 + \dfrac{\sigma\,\Delta t}{2\varepsilon}}, \qquad C_b = \frac{\dfrac{\Delta t}{\varepsilon}}{1 + \dfrac{\sigma\,\Delta t}{2\varepsilon}}$$

$C_a < 1$ introduces exponential damping (loss). For $\sigma = 0$ (lossless), $C_a = 1$ and $C_b = \Delta t / \varepsilon$.

Source Injection

We use a **soft source** (additive): $E_z[\mathrm{src}] \mathrel{+}= \mathrm{source\_value}$

This is preferred over a hard source ($E_z[\mathrm{src}] = \mathrm{source\_value}$) because:

- It does not create artificial reflections from the source point
- The source acts like a current density J_z driving the field

The **Ricker wavelet** (second derivative of a Gaussian) is used:

$$w(t) = \left(1 - 2\left(\pi f_c \tau\right)^2\right) \exp\left(-\left(\pi f_c \tau\right)^2\right), \qquad \tau = t - \frac{1}{f_c}$$

Properties: zero DC component, peaked at f_c, bandwidth $\approx 2.5\, f_c$.

Boundary Conditions

Without absorbing boundaries, waves reflect from domain edges (PEC — Perfect Electric Conductor — boundary by default).

CPML (Convolutional Perfectly Matched Layer):

- Creates an artificial absorbing layer at domain boundaries
- Impedance-matched at the interface $\Rightarrow$ zero theoretical reflection
- Exponential attenuation inside the layer
- Uses polynomial-graded conductivity: $\sigma(d) = \sigma_{\max}\left(\frac{d}{L}\right)^m$
- CFS (Complex Frequency-Shifted) variant adds κ and α parameters for wideband performance
- Implemented via recursive auxiliary variables (ψ fields)

Numerical Stability

CFL condition (Courant--Friedrichs--Lewy):

$$\Delta t \leq \frac{1}{c\sqrt{\frac{1}{\Delta x^2} + \frac{1}{\Delta z^2}}}$$

For a square grid ($\Delta x = \Delta z$):

$$\Delta t \leq \frac{\Delta x}{c\sqrt{2}}$$

Physical meaning: information cannot propagate more than one grid cell per time step. Violating this causes exponential field growth (instability).

Numerical dispersion: discrete waves travel slightly slower than continuous waves. Controlled by using ≥ 15--20 grid points per wavelength.

3.2 Adjoint-State FWI (Full Waveform Inversion) from EM Data

Objective Function

The least-squares misfit measures data discrepancy:

$$J(\mathbf{m}) = \frac{1}{2} \sum_{s,\, r,\, t} \left| d_{\text{obs}}(s, r, t) - d_{\text{syn}}(s, r, t;\, \mathbf{m}) \right|^2$$

where $\mathbf{m}$ is the model (e.g., ε_r distribution), d_{obs} is observed data, and d_{syn} is synthetic data from the forward simulation.

Why Direct Perturbation is Too Expensive

To compute $\partial J / \partial m_i$ for each of N model parameters:

- Finite-difference approach: perturb $m_i \to$ run forward simulation $\to$ compute ΔJ
- Cost: N **forward simulations** (one per parameter)
- For a 250×150 grid $= 37{,}500$ parameters $\to 37{,}500$ forward simulations
- Completely impractical!

How the Adjoint-State Method Computes Gradients Efficiently

The adjoint method uses the **Lagrangian formulation** with the PDE as a constraint:

$$\mathcal{L}(\mathbf{m},\, \mathbf{u},\, \boldsymbol{\lambda}) = J(\mathbf{u}) + \left\langle \boldsymbol{\lambda},\, F(\mathbf{m}, \mathbf{u}) \right\rangle$$

where $\mathbf{u}$ is the wavefield, $F(\mathbf{m}, \mathbf{u}) = 0$ is the wave equation, and $\boldsymbol{\lambda}$ is the Lagrange multiplier (adjoint field).

Setting $\partial \mathcal{L} / \partial \mathbf{u} = 0$ gives the **adjoint equation** — a wave equation with a special source. The gradient is then:

$$\frac{dJ}{d\mathbf{m}} = \frac{\partial \mathcal{L}}{\partial \mathbf{m}} = \left\langle \boldsymbol{\lambda},\ \frac{\partial F}{\partial \mathbf{m}}\,\mathbf{u} \right\rangle$$

Cost: only 2 simulations (one forward + one adjoint) regardless of the number of parameters!

Wavefield Meanings

Term	Description	How it is computed
Forward wavefield ($\mathbf{u}_{\text{fwd}}$)	Physical wave propagation from Tx through the current model	Standard FDTD forward simulation
Residual ($\mathbf{r}$)	Data discrepancy: $\mathbf{r} = d_{\text{syn}} - d_{\text{obs}}$	Difference of traces at Rx
Adjoint wavefield ($\mathbf{u}_{\text{adj}}$)	Sensitivity carrier: propagates time-reversed residual backward	FDTD simulation with time-reversed residual injected at Rx
Gradient ($\mathbf{g}$)	Sensitivity of J to model parameters at each grid point	Cross-correlation of forward and adjoint fields

Gradient Formula

For relative permittivity ε_r:

$$g_{\varepsilon_r}[i,j] = -\varepsilon_0 \sum_t \left\{ E_z^{\text{adj}}[i,j,t]\ \frac{\partial E_z^{\text{fwd}}[i,j,t]}{\partial t} \right\} \Delta t$$

Intuition:

- The forward field carries information about how the source illuminates each point
- The adjoint field carries information about how sensitive the data residual is to each point
- Their cross-correlation identifies where model changes would most reduce the misfit

Model Parameter Updates

1. Compute gradient $\mathbf{g}$ via the adjoint method
2. Choose search direction $\mathbf{d}$ (steepest descent: $\mathbf{d} = -\mathbf{g}$; L-BFGS: $\mathbf{d} \approx -\mathbf{H}^{-1}\mathbf{g}$)
3. Line search for step length α: find α that sufficiently reduces J
4. Update: $\mathbf{m}_{k+1} = \mathbf{m}_k + \alpha\,\mathbf{d}$
5. Apply bounds: $\varepsilon_r \in [1,\ 15]$
6. Add regularization (TV — Total Variation) to the gradient for sharp boundary recovery
7. Repeat until convergence

3.3 GPU/CUDA (Compute Unified Device Architecture) Acceleration

Which Operations to Move to GPU

The FDTD update equations are **embarrassingly parallel**: each grid cell's update depends only on its immediate neighbors. This maps perfectly to CUDA's SIMT (Single Instruction, Multiple Threads) model.

Operation	GPU suitability	Reasoning
$\mathbf{H}$-field update	Excellent	Each cell reads 2 values of E_z, writes 1 H value — independent per cell
$\mathbf{E}$-field update	Excellent	Each cell reads 2 values of H, writes 1 E value — independent per cell
CPML auxiliary updates	Good	Same stencil pattern, restricted to PML boundary regions
Source injection	Trivial	Single point write
Gradient accumulation	Excellent	Element-wise multiply-accumulate at every cell

Typical speedup: 30--80$\times$ over CPU implementations for FDTD.

What Stays on CPU

- **Parameter setup** (one-time cost, negligible)
- **Optimization loop control** (L-BFGS iterations, very lightweight)
- **I/O** (saving results to disk)

Practical Issues

Memory usage:

- Field arrays (E_z, H_x, H_y): $3 \times N_z \times N_x \times 8$ bytes ≈ 1.2 MB for our grid
- CPML auxiliary variables: ≈ 0.5 MB
- For adjoint: storing all forward fields $= N_t \times N_z \times N_x \times 8 \approx 770$ MB (fits on modern GPUs with ≥ 8 GB VRAM)
- Total for our problem: well within GPU memory limits

Memory access patterns:

- FDTD stencils access neighboring cells in both x and z directions

- **Row-major (C-order) storage** ensures that x-direction neighbors are contiguous in memory
- Adjacent threads should process adjacent x-indices $\rightarrow$ **coalesced memory access** $\rightarrow$ maximum bandwidth utilization
- Thread block size: typically 16×16 or 32×8, tuned to GPU architecture

CPU--GPU communication:

- **Minimize transfers**: keep all field arrays on GPU throughout the simulation
- **Only transfer receiver trace** back to CPU each step (single `float64` value — negligible overhead)
- **Transfer gradient** back to CPU once per inversion iteration (single $N_z \times N_x$ array)
- Avoid transferring entire field arrays unless needed for visualization/checkpointing

Implementation Options

1. **CuPy** (drop-in NumPy replacement): minimal code changes, same vectorized syntax
2. **Numba CUDA kernels**: explicit thread control, custom kernels for maximum performance
3. **PyTorch**: automatic differentiation for gradients, but overhead for small problems

For this project, CuPy is the best choice: it demonstrates GPU awareness without excessive implementation complexity.